
CSE323 — Software Engineering
Design Specification Report
Deliverable D3 — Phase 2
Sprint 2 Vertical Slice: UC-1 | UC-2 | UC-6 | UC-7
Sub-system: Customer Ordering System — Discovery & Identity

Submitted to:

Dr. Rami Zewail
Eng. Samy Shaawat
Eng. Rowida Elsayed

Team Members:

#	Name	Student ID
1	Zeyad Ashraf Tawfik Mohamed Attia	120230010
2	Ahmed Mohamed Rashed	120230263
3	Ahmed Youssef Kamal	120230255
4	Samir Ahmed	120230292
5	Hussein Sabri Youssef	120230244

Contents

1	Gherkin Scripts	2
1.1	Feature 1 — Browse Restaurant Catalog (UC-1)	2
1.2	Feature 2 — Explore Menu Details (UC-2)	4
1.3	Feature 3 — Register New Account (UC-6)	5
1.4	Feature 4 — Authenticate User Identity (UC-7)	7
2	QA Refinement Loop — Senior QA Audit Log	8
2.1	REQ19 — Server-Time Operating-Hours Validation	9
2.2	REQ5 — Password Complexity Guard	10
2.3	REQ6 — Rate-Limiting and Brute-Force Guard	10
2.4	REQ1 — Restaurant Catalog Metadata Display	11
2.5	REQ2 — Unavailable Menu Item State	12
3	API Contracts and Information Hiding	13
3.1	Endpoint 1 — Browse Restaurant Catalog	14
3.2	Endpoint 2 — Retrieve Restaurant Menu	15
3.3	Endpoint 3 — Register New Account	17
3.4	Endpoint 4 — Authenticate User Identity	19
4	UML Diagrams	20
4.1	Diagram 1: System Sequence Diagram — Authenticate User (UC-7)	21
4.2	Diagram 2: Activity Diagram — Browse Catalog & Menu (UC-1 & UC-2) . . .	23

1 Gherkin Scripts

The following scripts cover the complete Sprint 2 vertical slice (UC-1, UC-2, UC-6, UC-7). Every scenario is prefixed with a comment tagging the Requirement IDs it enforces, establishing a direct traceability link back to the D2 Requirements Report. **Scenario Outline** blocks are used wherever a guard must be verified across multiple input variations (REQ19 boundary cases, REQ5 password rules, REQ6 rate-limit thresholds).

1.1 Feature 1 — Browse Restaurant Catalog (UC-1)

Requirements covered: REQ1, REQ19

```

1 Feature: UC-1 -- Browse Restaurant Catalog
2   As an authenticated or guest user
3   I want to view a list of all active restaurants with their details
4   So that I can select a restaurant and begin building my order
5
6   Background:
7     Given the Customer Ordering System is running and reachable
8     And the database contains the following restaurant records:
9       | restaurant_id | name           | cuisine | avg_rating |
10      est_delivery_min | delivery_fee_egp | open_utc | close_utc |
11      | R001           | Burger Palace | American | 4.5       |
12      | 15.00           | 10:00         | 22:00   |
13      | R002           | Pizza Kingdom | Italian  | 4.2       |
14      | 20.00           | 11:00         | 23:00   |
15      | R003           | Sushi House   | Japanese | 4.8       |
16      | 25.00           | 12:00         | 21:00   |
17      | R004           | Night Bites   | Street   | 3.9       |
18      | 10.00           | 20:00         | 04:00   |
19
20   And no user session cart exists yet

```

Listing 1: UC-1 Feature Declaration and Background

```

1   # Covers: REQ1
2   Scenario: Display all open restaurants with required metadata fields
3     Given the server UTC time is "14:00"
4     When the user navigates to the homepage at "/"
5     Then the system shall return HTTP 200 for GET "/api/v1/restaurants"
6     And each restaurant entry in the UI shall display all of the
7       following fields:
8       | field           |
9       | name             |
10      | cuisine_category |
11      | avg_rating       |
12      | est_delivery_min |
13      | delivery_fee_egp |
14      | open_closed_status |
15
16     And the page shall render without a full page reload
17
18   # Covers: REQ1, REQ19
19   Scenario: Open restaurants are interactive; closed restaurants are
20     visually differentiated
21     Given the server UTC time is "14:00"
22     When the user navigates to the homepage
23     Then "Burger Palace" shall be displayed as interactive with status "
24     Open"
25     And "Night Bites" shall be displayed in a greyed-out non-interactive

```

```

state
22   with label "Currently Closed"
23   And the "Night Bites" card shall not be clickable

```

Listing 2: REQ1 Happy Path and REQ19 Visual Differentiation

```

1  # Covers: REQ19
2  Scenario: Closed restaurant is non-interactive even when client clock
    shows open time
3    Given "Burger Palace" has operating hours 10:00-22:00 server UTC
4    And the server UTC time is "03:00"
5    And the user's browser system clock is set to "14:00" (spoofed client
    time)
6    When the user navigates to the homepage
7    Then the server shall evaluate restaurant availability using its own
    UTC clock
8    And "Burger Palace" shall appear greyed-out with label "Currently
    Closed"
9    And clicking the "Burger Palace" card shall produce no navigation
    event
10   and no HTTP request

```

Listing 3: REQ19 Server-UTC Guard — Spoofed Client Clock Scenario

```

1  # Covers: REQ19
2  Scenario Outline: Browse-stage operating-hours gate enforced at exact
    UTC boundaries
3    Given "Burger Palace" has operating hours 10:00-22:00 server UTC
4    And the server UTC time is "<server_time>"
5    When the user loads the restaurant catalog
6    Then the "Burger Palace" card interactive state shall be "<
    interactive>"
7    And the displayed status label shall read "<label>"
8
9  Examples:
10   | server_time | interactive | label | description
11   | 09:59       | false      | Currently Closed | 1 minute before
    opening |
12   | 10:00       | true       | Open | exact opening
    boundary |
13   | 21:59       | true       | Open | last valid minute
14   | 22:00       | false      | Currently Closed | exact closing
    boundary |
15   | 03:00       | false      | Currently Closed | middle of the night
16
17 # Covers: REQ19
18 Scenario: Cross-midnight restaurant (Night Bites 20:00-04:00) is open
    at 23:00 UTC
19   Given "Night Bites" has operating hours 20:00-04:00 server UTC
20   And the server UTC time is "23:00"
21   When the user loads the restaurant catalog
22   Then "Night Bites" shall be displayed as interactive with status "
    Open"
23
24 # Covers: REQ19
25 Scenario: Cross-midnight restaurant is closed at 05:00 UTC
26   Given "Night Bites" has operating hours 20:00-04:00 server UTC

```

```

27   And the server UTC time is "05:00"
28   When the user loads the restaurant catalog
29   Then "Night Bites" shall be displayed as greyed-out with label "
    Currently Closed"

```

Listing 4: REQ19 Scenario Outline — Browse Boundary Cases

1.2 Feature 2 — Explore Menu Details (UC-2)

Requirements covered: REQ2, REQ19

```

1 Feature: UC-2 -- Explore Menu Details
2   As an authenticated or guest user
3   I want to view the full menu of a selected restaurant with item details
4   So that I can make an informed choice before adding items to my cart
5
6 Background:
7   Given the Customer Ordering System is running and reachable
8   And the server UTC time is within the operating window of "Burger
    Palace"
9   And the following menu items exist for "Burger Palace" (R001):
10      | item_id | category | name                                | price_egp |
    available |
11      | I001      | Burgers  | Classic Burger                      | 75.00      | true
    |
12      | I002      | Burgers  | Crispy Chicken Sandwich            | 85.00      | true
    |
13      | I003      | Specials | Unavailable Special                | 50.00      | false
    |
14      | I004      | Sides    | Loaded Fries                       | 45.00      | true
    |

```

Listing 5: UC-2 Feature Declaration and Background

```

1 # Covers: REQ2
2 Scenario: Display full menu organised by category with all required
    fields
3   Given the user clicks on the "Burger Palace" restaurant card
4   When the system calls GET "/api/v1/restaurants/R001/menu"
5   Then the server shall respond with HTTP 200
6   And the menu shall be organised by category sections: "Burgers" and "
    Sides"
7   And each available item tile shall display: name, description,
    price_egp, and availability_state
8   And no item description shall exceed 200 characters in the UI
9
10
11 # Covers: REQ2
12 Scenario: Unavailable item is displayed but non-interactive
13   Given the user is on the "Burger Palace" menu page
14   When the page finishes rendering
15   Then "Unavailable Special" (I003) shall be visible in its category
    section
16   And the "Unavailable Special" tile shall be in a greyed-out non-
    interactive state
17   And no click on the "Unavailable Special" tile shall dispatch a POST
18   to "/api/v1/cart/items"

```

Listing 6: REQ2 Happy Path and Unavailable Item Guard

```

1  # Covers: REQ19
2  Scenario: Menu endpoint returns HTTP 403 when restaurant is closed at
    server UTC time
3    Given "Burger Palace" has operating hours 10:00-22:00 server UTC
4    And the server UTC time is "03:00"
5    And the user's browser system clock is set to "15:00" (spoofed client
    time)
6    When the user requests GET "/api/v1/restaurants/R001/menu"
7    Then the server shall evaluate operating hours using its own UTC
    clock exclusively
8    And the server shall respond with HTTP 403 Forbidden
9    And the response body shall contain:
10     | field | value |
11     | code | RESTAURANT_CLOSED |
12     | message | Burger Palace is currently closed |
13     | server_utc_time_at_request | 03:00 |
14    And the menu item list shall not be included in the response

```

Listing 7: REQ19 Menu-View Checkpoint — Closed Restaurant Guard

```

1  # Covers: REQ19
2  Scenario Outline: Menu-view operating-hours gate enforced at exact UTC
    boundaries
3    Given "Burger Palace" has operating hours 10:00-22:00 server UTC
4    When a GET to "/api/v1/restaurants/R001/menu" arrives at server UTC "<server_time>"
5    Then the response status shall be <status>
6    And menu items shall be returned only if <menu_returned> is true
7
8  Examples:
9    | server_time | status | menu_returned | description |
10   | 09:59 | 403 | false | 1 minute before opening |
11   | 10:00 | 200 | true | exact opening boundary |
12   | 21:59 | 200 | true | last valid minute |
13   | 22:00 | 403 | false | exact closing boundary |
14   | 03:00 | 403 | false | middle of the night |

```

Listing 8: REQ19 Scenario Outline — Menu-View Boundary Cases

1.3 Feature 3 — Register New Account (UC-6)

Requirements covered: REQ5

```

1  Feature: UC-6 -- Register New Account
2  As a new visitor to the platform
3  I want to create an account using my email, password, and full name
4  So that I can place orders and access my order history
5
6  Background:
7    Given the Customer Ordering System is running and reachable
8    And the email "existing.user@example.com" already exists in the user
    database
9    And the SMS/Email Provider is operational and ready to dispatch
    emails
10
11  # Covers: REQ5
12  Scenario: Successfully register a new account with valid credentials
13    Given the user is on the registration page at "/register"

```

```

14  When the user enters:
15      | field      | value
16      | full_name   | Ahmed Mohamed Rashed
17      | email       | new.user@example.com
18      | password    | SecurePass1
19  And the user clicks "Create Account"
20  Then the server shall respond with HTTP 201 Created
21  And a new user record shall be created in the database
22  And the SMS/Email Provider shall receive exactly one dispatch request
23  And the UI shall display: "Please check your email to verify your
    account."
24  And no JWT session token shall be issued until email is verified

```

Listing 9: UC-6 Feature Declaration and Happy Path

```

1  # Covers: REQ5
2  Scenario: UI disables submit while password is fewer than 8 characters
3  Given the user is on the registration page
4  When the user enters "Short1" into the password field (6 characters)
5  Then the "Create Account" button shall be disabled
6  And a UI message shall read "Password must be at least 8 characters"
7  And no HTTP request shall be dispatched to "/api/v1/auth/register"
8
9  # Covers: REQ5
10 Scenario: UI disables submit while password contains no numeral
11 Given the user is on the registration page
12 When the user enters "NoNumbersHere" into the password field
13 Then the "Create Account" button shall be disabled
14 And a UI message shall read "Password must contain at least one
    number"
15 And no HTTP request shall be dispatched to "/api/v1/auth/register"

```

Listing 10: REQ5 Password Complexity Guards at UI and API Layers

```

1  # Covers: REQ5
2  Scenario Outline: Server rejects non-compliant passwords with HTTP 422
3  Given a client bypasses the UI and sends POST to "/api/v1/auth/
    register"
4  When the request body contains email "test@test.com" and password "<
    password>"
5  Then the server shall respond with HTTP 422 Unprocessable Entity
6  And the response error code shall be "VALIDATION_ERROR"
7  And the response fields.password shall equal "<error_message>"
8  And no user record shall be created in the database
9  And no verification email shall be dispatched
10
11 Examples:
12 | password      | error_message
13 | short1        | Password must be at least 8 characters
14 | Short         | Password must be at least 8 characters
15 | nonnumeralpass | Password must contain at least one number
16 | NONNUMERALUPPER | Password must contain at least one number
17 | 1234567       | Password must be at least 8 characters
18

```

```

19 # Covers: REQ5
20 Scenario: Server rejects registration with a duplicate email address
21   When a POST is sent to "/api/v1/auth/register" with email
22     "existing.user@example.com" and password "ValidPass9"
23   Then the server shall respond with HTTP 409 Conflict
24   And the response error code shall be "EMAIL_ALREADY_EXISTS"
25   And no new user record shall be created and no email shall be
    dispatched

```

Listing 11: REQ5 Scenario Outline — Server-Side Password Validation

1.4 Feature 4 — Authenticate User Identity (UC-7)

Requirements covered: REQ6

```

1 Feature: UC-7 -- Authenticate User Identity
2   As a registered user
3   I want to log in with my email and password
4   So that I can access my account, order history, and place orders
5
6 Background:
7   Given the Customer Ordering System is running and reachable
8   And a verified user account exists with email "verified.user@example.
9     com"
10    and password "CorrectPass1" (hashed) and status EMAIL_VERIFIED
11   And no prior failed login attempts exist in the current 10-minute
    window
12
13 # Covers: REQ6
14 Scenario: Successfully authenticate with correct credentials
15   Given the user is on the login page at "/login"
16   When the user enters email "verified.user@example.com" and password "
17     CorrectPass1"
18   And the user clicks "Log In"
19   Then the server shall respond with HTTP 200 OK
20   And the response shall include a signed JWT delivered via HTTP-only
    Set-Cookie header
21   And the JWT token shall have a configurable expiry present in its
    payload
22   And the user shall be redirected to the homepage at "/"

```

Listing 12: UC-7 Feature Declaration and Happy Path

```

1 # Covers: REQ6
2 Scenario: Account locked after 5 consecutive failed attempts within 10
    minutes
3   Given the account has 4 consecutive failed attempts in the current
    10-minute window
4   When the user submits a 5th failed attempt with password "
5     WrongPassword9"
6   Then the server shall respond with HTTP 429 Too Many Requests
7   And the response error code shall be "ACCOUNT_LOCKED"
8   And the message shall read "Too many failed attempts. Please try
9     again later."
10   And no JWT token shall be issued
11
12 # Covers: REQ6
13 Scenario: Locked account rejects even the correct password during the
    lockout window

```

```

12  Given the account has been locked due to 5 consecutive failures
13  And the 10-minute lockout window has NOT yet elapsed
14  When the user submits the correct password "CorrectPass1"
15  Then the server shall respond with HTTP 429 Too Many Requests
16  And no JWT token shall be issued
17
18  # Covers: REQ6
19  Scenario: Successful login resets the consecutive failure counter to
      zero
20  Given the account has 3 consecutive failed login attempts
21  When the user submits a successful login with password "CorrectPass1"
22  Then the server shall respond with HTTP 200 OK
23  And the consecutive failed attempt counter shall be reset to 0
24  And a valid JWT session token shall be issued

```

Listing 13: REQ6 Rate-Limiting Guard — Lockout and Boundary Scenarios

```

1  # Covers: REQ6
2  Scenario Outline: Rate-limiting lockout boundary enforced at exactly 5
      attempts
3  Given the account has <prior_failures> consecutive failures in the
      current window
4  When the user submits another failed attempt with password "
      WrongPassword9"
5  Then the server shall respond with HTTP <expected_status>
6  And the response error code shall be "<expected_code>"
7
8  Examples:
9  | prior_failures | expected_status | expected_code |
      description |
10 | 0               | 401             | INVALID_CREDENTIALS | first
      failure -- no lockout |
11 | 1               | 401             | INVALID_CREDENTIALS | second
      failure -- no lockout |
12 | 3               | 401             | INVALID_CREDENTIALS | fourth
      failure -- no lockout |
13 | 4               | 429             | ACCOUNT_LOCKED      | fifth
      failure -- lockout triggers |
14 | 5               | 429             | ACCOUNT_LOCKED      | already
      locked -- remains locked |
15
16  # Covers: REQ6
17  Scenario: Non-existent email returns same HTTP 401 as wrong password
18  When a POST is sent to "/api/v1/auth/login" with email "ghost@nowhere
      .com"
19  and password "AnyPass1"
20  Then the server shall respond with HTTP 401 Unauthorized
21  And the response error code shall be "INVALID_CREDENTIALS"
22  And no account-specific information shall be revealed in the response

```

Listing 14: REQ6 Scenario Outline — Rate-Limit Boundary Enforcement

2 QA Refinement Loop — Senior QA Audit Log

This section documents the mandatory refinement pass executed before any Gherkin scenario was written. Each subsection presents one requirement that contained an unquantifiable term in its original draft, the QA critique explaining why the draft could not be automated, and the

final measurable metric adopted in the Gherkin scripts.

The governing principle is identical across all cases: **Playwright can only assert boolean outcomes**. A test can assert that a button has `disabled=true` or that a response carries status code 429; it cannot assert that a UI “looks nice,” that a password is “secure,” or that the system has “stopped” an attacker. Every vague adjective is a gap in the test contract.

2.1 REQ19 — Server-Time Operating-Hours Validation

Feature scope: UC-1 (Browse), UC-2 (Menu View)

Original Draft (Vague Phrasing)

“The system should show whether a restaurant is currently open or closed.”

QA Critique — Why It Is Untestable

“**Show**” defines no authoritative time source, no HTTP response code, and no DOM state. A client-side `new Date()` call satisfies this phrasing while being trivially bypassable by any user who changes their system clock. There is no boolean outcome for Playwright to assert against — “shown as open” is a visual impression, not a testable condition. No boundary times, no HTTP status code for rejection, and no definition of which clock governs are provided.

Final Measurable Metric

The server evaluates `open_time_utc <= server_utc_now() < close_time_utc` exclusively using its own UTC clock. At the **browse checkpoint (UC-1)**: closed restaurants render with `is_open=false`, label “Currently Closed”, and zero HTTP requests fire on click. At the **menu-view checkpoint (UC-2)**: a GET outside operating hours returns **HTTP 403** with code: “RESTAURANT_CLOSED” and `server_utc_time_at_request` in the body. Boundary table verified at exactly five points: 09:59→403, 10:00→200, 21:59→200, 22:00→403, 03:00→403.

```

1  # Covers: REQ19
2  Scenario Outline: Menu-view operating-hours gate enforced at exact UTC
   boundaries
3    Given "Burger Palace" has operating hours 10:00-22:00 server UTC
4    When a GET to "/api/v1/restaurants/R001/menu" arrives at server UTC "<server_time>"
5    Then the response status shall be <status>
6    And menu items shall be returned only if <menu_returned> is true
7
8  Examples:
9    | server_time | status | menu_returned | description |
10   | 09:59       | 403    | false         | 1 minute before opening |
11   | 10:00       | 200    | true          | exact opening boundary |
12   | 21:59       | 200    | true          | last valid minute |
13   | 22:00       | 403    | false         | exact closing boundary |
14   | 03:00       | 403    | false         | middle of the night |

```

Listing 15: REQ19 Refined Metric — Scenario Outline as Used in Gherkin Scripts

Metric adopted: Operating-hours eligibility evaluated exclusively on the server’s UTC clock. HTTP 403 with “RESTAURANT_CLOSED” for out-of-hours requests. Playwright test spoofs client clock to 14:00 while server UTC is 03:00; expected outcome is HTTP 403 regardless.

2.2 REQ5 — Password Complexity Guard

Feature scope: UC-6 (Register New Account)

Original Draft (Vague Phrasing)

“Users should choose a secure password when registering.”

QA Critique — Why It Is Untestable

“**Secure**” is an architectural aspiration with no operational definition. Two developers reading this independently will implement different rules — one may require a symbol, another may set a minimum of six characters. There is no integer boundary, no regex pattern, no HTTP status code, and no specific error string. A password of “a” could be argued to satisfy “secure” in a sufficiently lenient reading. The phrasing passes code review while leaving the constraint entirely unimplemented.

Final Measurable Metric

Password must satisfy **both** conditions simultaneously: (1) minimum length of **8 characters**, and (2) at least **one numeric digit (0–9)**. At the **UI layer**: the submit button transitions to `disabled=true` the moment either condition is unmet. At the **API layer**: any violation returns **HTTP 422** with code: “`VALIDATION_ERROR`” and the exact `fields.password` message. No user record is created and no email is dispatched on failure. Verified across five distinct Scenario Outline rows.

```

1  # Covers: REQ5
2  Scenario Outline: Server rejects non-compliant passwords with HTTP 422
3    Given a client bypasses the UI and sends POST to "/api/v1/auth/
4    register"
5    When the request body contains email "test@test.com" and password "<
6    password>"
7    Then the server shall respond with HTTP 422 Unprocessable Entity
8    And the response error code shall be "VALIDATION_ERROR"
9    And the response fields.password shall equal "<error_message>"
10   And no user record shall be created and no verification email shall
11   be dispatched
12
13 Examples:
14   | password          | error_message                                     |
15   | short1             | Password must be at least 8 characters           |
16   | nonumeralpass      | Password must contain at least one number       |
17   | NONUMERALUPPER     | Password must contain at least one number       |
18   | 1234567            | Password must be at least 8 characters           |

```

Listing 16: REQ5 Refined Metric — Scenario Outline as Used in Gherkin Scripts

Metric adopted: Quantity field must satisfy: $\text{length} \geq 8$ AND $\exists d \in \{0 \dots 9\}$ in the string. UI: button `disabled=true` while either condition fails. API: violation returns HTTP 422 with exact message string.

2.3 REQ6 — Rate-Limiting and Brute-Force Guard

Feature scope: UC-7 (Authenticate User Identity)

Original Draft (Vague Phrasing)

“The system should stop hackers from guessing passwords too many times.”

QA Critique — Why It Is Untestable

“**Hackers**” is not an actor the test runner can instantiate. “**Too many times**” defines no integer threshold and no time window — one developer implements a limit of 3, another of 100. “**Stop**” defines no HTTP response code, no lockout duration, and no scope (per-account? per-IP? global?). There is no boolean outcome Playwright can assert: “stopped” is not an observable DOM state or response field.

Final Measurable Metric

After exactly **5 consecutive failed login attempts** for a single account within a **rolling 10-minute window**, all further requests for that account — including requests containing the **correct password** — return **HTTP 429** with code: “ACCOUNT_LOCKED”. The counter is **scoped per account**. A successful login **resets the counter to zero**. Window expiry resets the counter and the next failure returns HTTP 401 (not 429). Boundary enforced via Scenario Outline: 4 prior failures + 1 new → HTTP 429; 3 prior + 1 new → HTTP 401.

```

1  # Covers: REQ6
2  Scenario Outline: Rate-limiting lockout boundary enforced at exactly 5
   attempts
3    Given the account has <prior_failures> consecutive failures in the
   current window
4    When the user submits another failed attempt with password "
   WrongPassword9"
5    Then the server shall respond with HTTP <expected_status>
6    And the response error code shall be "<expected_code>"
7
8  Examples:
9    | prior_failures | expected_status | expected_code |
10   | 0               | 401             | INVALID_CREDENTIALS |
11   | 3               | 401             | INVALID_CREDENTIALS |
12   | 4               | 429             | ACCOUNT_LOCKED      |
13   | 5               | 429             | ACCOUNT_LOCKED      |

```

Listing 17: REQ6 Refined Metric — Scenario Outline as Used in Gherkin Scripts

Metric adopted: Counter is per-account, integer-bounded at 5, within a 10-minute rolling window. HTTP 429 is the only possible response once the threshold is reached, regardless of password correctness.

2.4 REQ1 — Restaurant Catalog Metadata Display

Feature scope: UC-1 (Browse Restaurant Catalog)

Original Draft (Vague Phrasing)

“The homepage should nicely show all the restaurants available.”

QA Critique — Why It Is Untestable

“**Nicely**” is a subjective aesthetic judgement with no DOM-queryable definition. “**Available**” is undefined — available to whom, and under what server-time condition? The phrasing specifies neither which data fields must be present nor how closed restaurants differ from open ones visually. A page rendering a single restaurant name satisfies this phrasing. There is no field checklist, no state assertion, and no network-level condition.

Final Measurable Metric

The catalog API must return **HTTP 200** and each restaurant entry must expose **all six fields**: `name`, `cuisine_category`, `avg_rating`, `est_delivery_min`, `delivery_fee_egp`, and `open_closed_status`. Open restaurants render as **interactive** (clickable). Closed restaurants render as **non-interactive** with the exact label “Currently Closed” and zero HTTP requests fire on click. Page renders **without a full page reload** (no `DOMContentLoaded` event fires during fetch).

```

1  # Covers: REQ1
2  Scenario: Display all open restaurants with required metadata fields
3  Given the server UTC time is "14:00"
4  When the user navigates to the homepage at "/"
5  Then the system shall return HTTP 200 for GET "/api/v1/restaurants"
6  And each restaurant entry in the UI shall display all of the
   following:
7      | field                |
8      | name                  |
9      | cuisine_category      |
10     | avg_rating            |
11     | est_delivery_min      |
12     | delivery_fee_egp      |
13     | open_closed_status    |
14  And the page shall render without a full page reload

```

Listing 18: REQ1 Refined Metric — Field Completeness and Interaction State

Metric adopted: Six mandatory fields asserted via DOM attribute queries. `is_open=false` cards assert `pointer-events: none` or `aria-disabled="true"` and zero network requests on click.

2.5 REQ2 — Unavailable Menu Item State

Feature scope: UC-2 (Explore Menu Details)

Original Draft (Vague Phrasing)

“Menu items that are sold out should look different so users don’t click them.”

QA Critique — Why It Is Untestable

“**Look different**” defines no CSS class, no DOM attribute, no aria state, and no interaction contract. It is a visual impression that Playwright cannot assert without brittle pixel-comparison tests. “**Don’t click them**” defines no enforcement mechanism — a greyed-out item that still fires a POST on click technically satisfies “looks different.” There is no API-level guard and no HTTP response code specified.

Final Measurable Metric

Any item with `available=false` must: (1) render with `aria-disabled="true"` or `disabled=true` on the tile, (2) have **no active “Add to Cart” button**, and (3) dispatch **zero POST requests** to `/api/v1/cart/items` regardless of any click event. At the API layer, the server re-validates availability on every cart-add, returning **HTTP 422** with code: `"ITEM_UNAVAILABLE"` if bypassed.

```

1  # Covers: REQ2
2  Scenario: Unavailable item is displayed but non-interactive
3    Given the user is on the "Burger Palace" menu page
4    When the page finishes rendering
5    Then "Unavailable Special" (I003) shall be visible in its category
    section
6    And the "Unavailable Special" tile shall be in a greyed-out non-
    interactive state
7    And no click on the "Unavailable Special" tile shall dispatch
8    a POST to "/api/v1/cart/items"

```

Listing 19: REQ2 Refined Metric — Unavailable Item Interaction Guard

Metric adopted: Availability enforced at both DOM layer (`aria-disabled="true"`) and API layer (HTTP 422 on bypass). Zero POST requests fire for unavailable items — asserted via `page.route()` interception in Playwright.

QA Lead’s Note: This refinement pass was a mandatory prerequisite before a single line of Gherkin was authored. Every vague adjective in a draft requirement is a gap in the test contract: it documents an *intention* while leaving the implementation unconstrained. The refinement process replaces each vague term with three concrete artefacts: (1) an exact integer or string boundary, (2) an observable system state (HTTP status code, DOM attribute, network request count), and (3) a negative-case assertion. Only once all three artefacts exist is it possible to write a **Scenario Outline** that a CI pipeline can evaluate deterministically.

3 API Contracts and Information Hiding

Table 1: Global API Conventions

Convention	Value
Base URL	<code>/api/v1</code>
Content-Type	<code>application/json</code>
Authentication	HTTP-only session cookie (<code>session_id</code>) where required
Monetary values	number with exactly 2 decimal places, in EGP
Timestamps	ISO 8601 UTC strings: <code>"2026-05-10T14:32:00Z"</code>
Rating values	number with exactly 1 decimal place, range 0.0–5.0
Error envelope	Uniform structure across all endpoints (see below)

```

1  {
2    "error": {

```

```

3     "code": "MACHINE_READABLE_CODE",
4     "message": "Human-readable description of the error.",
5     "fields": {
6         "field_name": "Field-specific detail (present only for validation
        errors)"
7     }
8 }
9 }

```

Listing 20: Standard Error Envelope — applies to all non-2xx responses

3.1 Endpoint 1 — Browse Restaurant Catalog

Property	Value
Method & URL	GET /api/v1/restaurants
Maps to	UC-1: Browse Restaurant Catalog
Enforces	REQ1, REQ19
Authentication	Not required (public endpoint)

Purpose. Returns the complete list of all active restaurants. Each entry includes the full metadata required by REQ1 and an `is_open` boolean computed exclusively from the server’s UTC clock at the moment of the request. The client receives a static snapshot and never participates in the open/closed determination.

No request body or query parameters are required.

```

1 {
2     "restaurants": [
3         {
4             "restaurant_id": "R001",
5             "name": "Burger Palace",
6             "cuisine_category": "American",
7             "avg_rating": 4.5,
8             "est_delivery_min": 25,
9             "delivery_fee_egp": 15.00,
10            "is_open": true,
11            "status_label": "Open",
12            "operating_hours_display": "10:00 AM - 10:00 PM"
13        },
14        {
15            "restaurant_id": "R002",
16            "name": "Pizza Kingdom",
17            "cuisine_category": "Italian",
18            "avg_rating": 4.2,
19            "est_delivery_min": 35,
20            "delivery_fee_egp": 20.00,
21            "is_open": false,
22            "status_label": "Currently Closed",
23            "operating_hours_display": "11:00 AM - 11:00 PM"
24        }
25    ],
26    "total_count": 2,
27    "server_utc_time_at_request": "2026-05-10T03:00:00Z"
28 }

```

Listing 21: HTTP 200 — Catalog Retrieved Successfully

REQ19 Contract Invariant: `is_open` is always server-computed. There is no client-supplied parameter that can influence its value. `server_utc_time_at_request` is surfaced for debugging transparency only and must never be used as input to any client-side logic.

```

1 {
2   "error": {
3     "code": "SERVICE_UNAVAILABLE",
4     "message": "The restaurant catalog is temporarily unavailable. Please
      try again."
5   }
6 }

```

Listing 22: HTTP 503 — Service Unavailable

Information Hiding Rationale

- **Operating-hours computation** — the server evaluates the UTC window comparison (including cross-midnight logic) entirely internally. The client receives one boolean: `is_open`.
- **Database schema** — storage engine, table names, and index structure are invisible to the client.
- **Rating aggregation** — `avg_rating` is a pre-computed aggregate. Whether it is computed on-the-fly or cached is opaque.
- **operating_hours_display** — a pre-formatted string. The raw `open_time_utc` and `close_time_utc` column values are never exposed, preventing client-side time comparisons.

3.2 Endpoint 2 — Retrieve Restaurant Menu

Property	Value
Method & URL	GET <code>/api/v1/restaurants/{id}/menu</code>
Maps to	UC-2: Explore Menu Details
Enforces	REQ2, REQ19
Authentication	Not required (public endpoint)

Purpose. Returns the full menu for a single restaurant organised by category. The server enforces REQ19 at this checkpoint: if the restaurant is outside its operating window, the endpoint returns HTTP 403 and no menu data is included. The client supplies only the `restaurant_id` path parameter.

```

1 {
2   "restaurant": {
3     "restaurant_id": "R001",
4     "name": "Burger Palace",
5     "is_open": true
6   },
7   "menu": [
8     {
9       "category": "Burgers",
10      "items": [
11        {
12          "item_id": "I001",

```

```
13     "name": "Classic Burger",
14     "description": "Beef patty, lettuce, tomato, house sauce.",
15     "price_egg": 75.00,
16     "available": true
17   },
18   {
19     "item_id": "I003",
20     "name": "Unavailable Special",
21     "description": "Chef's secret recipe.",
22     "price_egg": 50.00,
23     "available": false
24   }
25 ]
26 },
27 {
28   "category": "Sides",
29   "items": [
30     {
31       "item_id": "I004",
32       "name": "Loaded Fries",
33       "description": "Fries with cheese and jalapenos.",
34       "price_egg": 45.00,
35       "available": true
36     }
37   ]
38 }
39 ],
40 "server_utc_time_at_request": "2026-05-10T14:00:00Z"
41 }
```

Listing 23: HTTP 200 — Menu Retrieved Successfully

```
1 {
2   "error": {
3     "code": "RESTAURANT_CLOSED",
4     "message": "Burger Palace is currently closed and cannot accept
5     orders.",
6     "details": {
7       "restaurant_id": "R001",
8       "operating_hours_utc": {
9         "open": "10:00",
10        "close": "22:00"
11      },
12      "server_utc_time_at_request": "03:00"
13    }
14 }
```

Listing 24: HTTP 403 — Restaurant Closed at Server UTC Time (REQ19)

```
1 {
2   "error": {
3     "code": "RESTAURANT_NOT_FOUND",
4     "message": "No restaurant exists with the provided ID."
5   }
6 }
```

Listing 25: HTTP 404 — Restaurant Not Found

Information Hiding Rationale

- **Operating-hours gate** — the client receives HTTP 403 and reacts; the time comparison algorithm is hidden. There is no request field for client-supplied time, making the time authority architecturally unambiguous.
- **Item availability mechanism** — the client has no knowledge of how `available: false` is set. The interface exposes only the outcome.
- **Price authority** — `price_egg` values here are display data. The cart and checkout endpoints re-query prices independently on every write operation.
- **Category ordering** — sort order is determined entirely server-side; the client renders the array in received order without sorting logic.

3.3 Endpoint 3 — Register New Account

Property	Value
Method & URL	POST <code>/api/v1/auth/register</code>
Maps to	UC-6: Register New Account
Enforces	REQ5
Authentication	Not required

Purpose. Creates a new user account. The server enforces all REQ5 constraints before persisting any record. On success, it dispatches a verification email and returns HTTP 201. No session token is issued until email verification is complete.

```

1 {
2   "full_name": "string",    // Required. 1-100 characters.
3   "email":      "string",    // Required. Valid RFC 5322 format. Must be
    unique.
4   "password":  "string"      // Required. REQ5: min 8 chars, >= 1 numeric
    digit.
5                                   // Immediately hashed server-side. Never
    stored/logged
6                                   // in plaintext. Never returned in any
    response.
7 }
```

Listing 26: Request Payload — POST `/api/v1/auth/register`

```

1 {
2   "user": {
3     "user_id":      "USR-20260510-00192",
4     "email":        "new.user@example.com",
5     "full_name":    "Ahmed Mohamed Rashed",
6     "status":       "PENDING_VERIFICATION",
7     "created_at":   "2026-05-10T14:32:00Z"
8   },
9   "message": "Please check your email to verify your account."
10 }
```

Listing 27: HTTP 201 — Account Created Successfully

```
1 {
2   "error": {
3     "code": "EMAIL_ALREADY_EXISTS",
4     "message": "An account with this email address already exists."
5   }
6 }
```

Listing 28: HTTP 409 — Email Address Already Registered

```
1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "Request payload failed validation.",
5     "fields": {
6       "password": "Password must be at least 8 characters and contain at
7         least one number."
8     }
9   }
10 }
```

Listing 29: HTTP 422 — Password Complexity Failure (REQ5)

```
1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "Request payload failed validation.",
5     "fields": {
6       "email": "Must be a valid email address.",
7       "password": "Password must be at least 8 characters and contain at
8         least one number."
9     }
10 }
```

Listing 30: HTTP 422 — Multiple Field Failures

Information Hiding Rationale

- **Password hashing** — the hashing algorithm, salt rounds, and pepper are entirely internal. The interface boundary is: plaintext in, nothing out.
- **Email verification mechanism** — token format, expiry duration, and retry logic are fully opaque to the client.
- **User ID generation** — `user_id` is server-generated; the client cannot predict or supply it.
- **Duplicate detection** — HTTP 409 communicates the outcome; the query strategy and case-sensitivity normalisation are hidden.

3.4 Endpoint 4 — Authenticate User Identity

Property	Value
Method & URL	POST /api/v1/auth/login
Maps to	UC-7: Authenticate User Identity
Enforces	REQ6
Authentication	Not required (this endpoint establishes authentication)

Purpose. Validates submitted credentials and issues a signed JWT via HTTP-only cookie on success. The server enforces REQ6’s rate-limiting invariant: after 5 consecutive failed attempts within a rolling 10-minute window, all requests for that account return HTTP 429 until the window elapses. The failure counter is scoped per account. A successful login resets the counter to zero.

```

1 {
2   "email":      "string",    // Required. Non-empty. No format validation
3                                   // (intentionally generic to prevent user
4   "password":  "string"      // Required. Non-empty. No complexity check at
5                                   // stage -- credential matching only.
6 }

```

Listing 31: Request Payload — POST /api/v1/auth/login

```

1 {
2   "user": {
3     "user_id":   "USR-20260510-00192",
4     "email":     "verified.user@example.com",
5     "full_name": "Ahmed Mohamed Rashed"
6   },
7   "token": {
8     "expires_at": "2026-05-11T14:32:00Z"
9   }
10 }
11 // NOTE: The JWT is delivered ONLY via HTTP-only Set-Cookie header.
12 // It is never present in the JSON response body.
13 // expires_at is for UI display only; authoritative expiry is in the
    signed token.

```

Listing 32: HTTP 200 — Authentication Successful

```

1 {
2   "error": {
3     "code": "INVALID_CREDENTIALS",
4     "message": "Invalid email or password."
5   }
6 }
7 // SECURITY: Response is structurally identical whether the email does
    not exist
8 // or the password is incorrect. Divergent responses enable user
    enumeration.

```

Listing 33: HTTP 401 — Invalid Credentials

```
1 {
2   "error": {
3     "code": "ACCOUNT_LOCKED",
4     "message": "Too many failed attempts. Please try again later.",
5     "details": {
6       "lockout_expires_at": "2026-05-10T14:42:00Z"
7     }
8   }
9 }
10 // REQ6 INVARIANT: HTTP 429 is returned for ALL requests against a locked
    account,
11 // including requests with the correct password, until lockout_expires_at
    is reached.
```

Listing 34: HTTP 429 — Account Locked: Rate Limit Exceeded (REQ6)

```
1 {
2   "error": {
3     "code": "EMAIL_NOT_VERIFIED",
4     "message": "Please verify your email address before logging in."
5   }
6 }
```

Listing 35: HTTP 403 — Account Not Yet Verified

Information Hiding Rationale

- **JWT signing mechanism** — the signing algorithm, secret key, and token rotation strategy are entirely internal. The interface boundary is: valid credentials in, authenticated session out via cookie.
- **Password verification** — hash comparison is opaque. HTTP 401 is the only observable outcome of a mismatch.
- **Rate-limit counter storage** — whether the counter is stored in Redis, an in-memory cache, or a database row is hidden. The scope (per account, not per IP) is observable only through the HTTP 429 response.
- **Lockout window computation** — `lockout_expires_at` is server-computed. The client displays it; it does not compute window boundaries.
- **User enumeration prevention** — HTTP 401 for both non-existent email and wrong password. The internal branching logic is fully hidden behind one uniform response contract.

4 UML Diagrams

Rubric Criterion: Correct System Sequence Diagrams (SSD) showing happy and failure paths, and an Activity Diagram with explicit code decision points. Notation must be accurate.

4.1 Diagram 1: System Sequence Diagram — Authenticate User (UC-7)

Engineering Description

This SSD models the complete server-side execution pipeline for UC-7: Authenticate User Identity, capturing the interaction between three participants: the **Customer (Browser)**, the **System (API Server)**, and the **Database**.

The diagram is structured as a cascading **alt/else** block that evaluates four sequential gates in the exact order the server processes them:

1. **Gate 1 (REQ6)** — Rate-limit pre-check. The server queries the failure counter *before* any credential lookup. If `failed.attempts` ≥ 5 within the current 10-minute window, the request is immediately rejected with HTTP 429, regardless of password correctness. This gate is evaluated first to prevent unnecessary database credential lookups on locked accounts.
2. **Gate 2** — Account lookup by email. A null result returns HTTP 401 with the same `INVALID_CREDENTIALS` body as a wrong-password response, preventing user enumeration.
3. **Gate 3** — Email verification status check. An account with `status = "PENDING_VERIFICATION"` returns HTTP 403 before any password comparison is attempted.
4. **Gate 4** — Password hash verification. A mismatch increments the failure counter and returns HTTP 401. A match resets the counter to zero, generates a signed JWT delivered exclusively via HTTP-only **Set-Cookie** header, and returns HTTP 200. The REQ6 reset is annotated on this terminal success response.

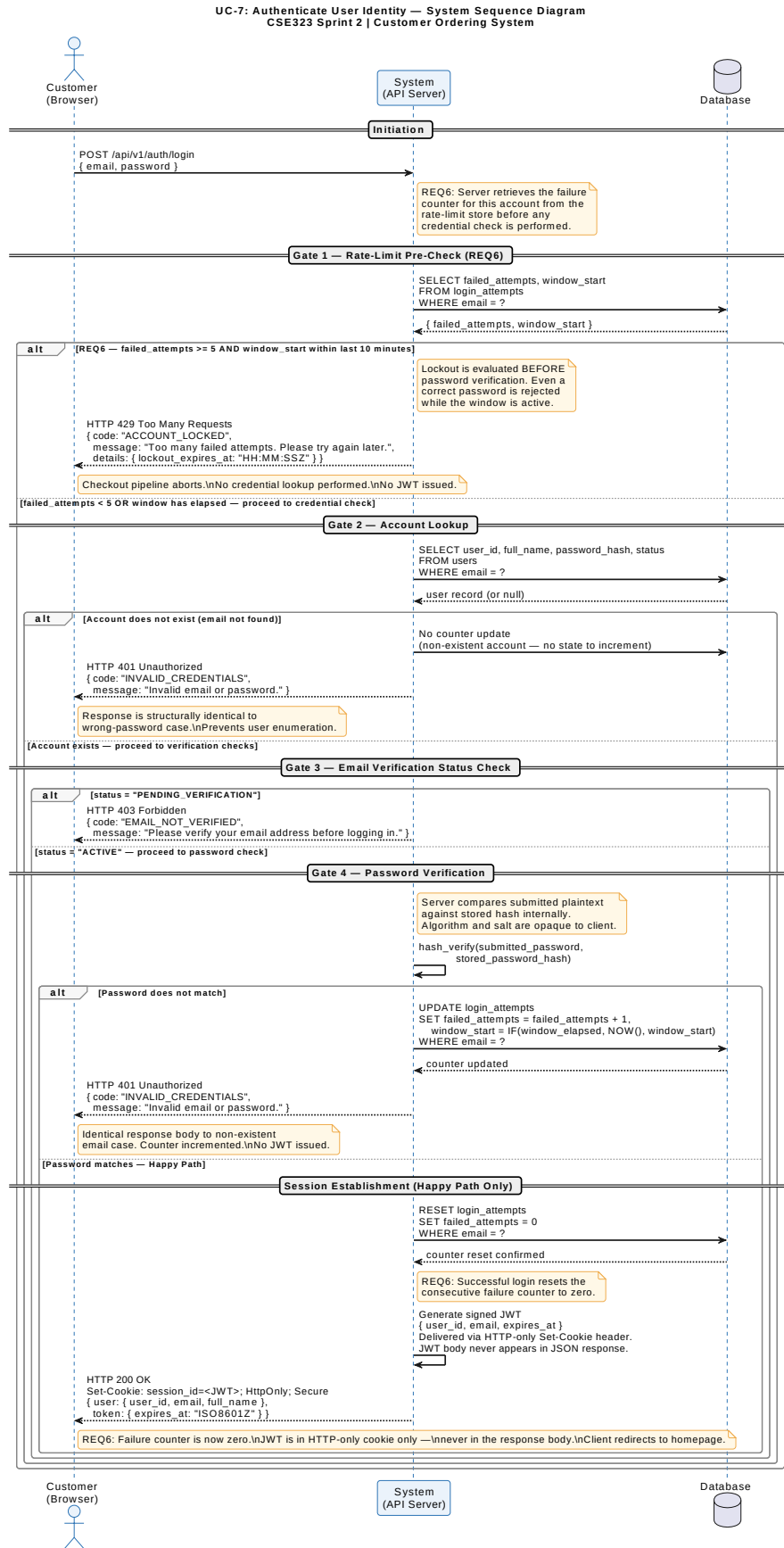


Figure 1: SSD — Authenticate User Identity (UC-7): Happy Path and all Failure Paths

4.2 Diagram 2: Activity Diagram — Browse Catalog & Menu (UC-1 & UC-2)

Engineering Description

This Activity Diagram models the complete server-side execution logic for UC-1: Browse Restaurant Catalog and UC-2: Explore Menu Details, structured across two swim lanes — **Browser (Client)** and **API Server** — with Database operations annotated as interaction steps within the server lane.

Four explicit decision diamonds correspond to the core guards from the D2 Requirements Report. Each guard appears in the exact order the server evaluates it, forming an early-exit chain:

1. **Padlock 1 (REQ19) — Browse Checkpoint.** For each restaurant record returned by the database, the server evaluates `open_time_utc ≤ server_utc_now() < close_time_utc` using its own UTC clock exclusively. A cross-midnight check (e.g., Night Bites 20:00–04:00) is handled via a separate branch. The result sets `is_open` to `true` or `false` and `status_label` to "Open" or "Currently Closed". The client never participates in this computation.
2. **Padlock 2 (REQ19) — Menu-View Checkpoint.** When the user selects an open restaurant, the server independently re-captures `server_utc_now()` for a fresh evaluation. The `is_open` value from the catalog response is *not* reused. An out-of-hours request at this stage returns HTTP 403 with code: "RESTAURANT_CLOSED" and `server_utc_time_at_request` in the body.
3. **Padlock 3 (REQ2) — Item Availability Check.** Each menu item is evaluated against its live `available` flag in the database. Items with `available = true` are flagged as interactive; items with `available = false` are included in the response but flagged as non-interactive, enabling the UI to render them greyed-out without a separate follow-up request.
4. **Restaurant Not Found Guard.** A `restaurant_id` that does not exist in the database returns HTTP 404 before any operating-hours logic is evaluated.

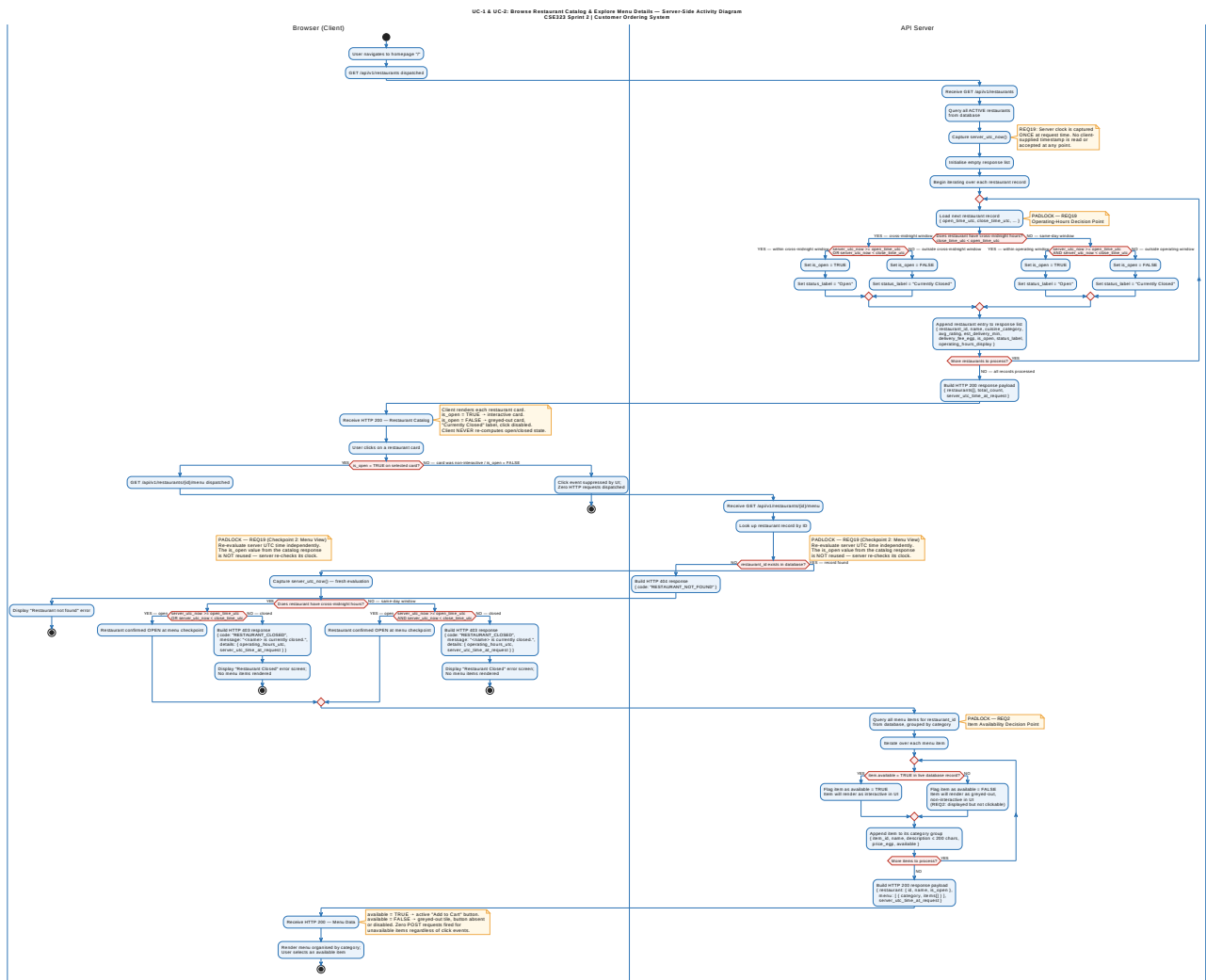


Figure 2: Activity Diagram — Browse Restaurant Catalog & Explore Menu Details (UC-1 & UC-2): Server-Side Logic showing REQ19 and REQ2 Padlock Guards as explicit decision diamonds